

Make `<compare>` freestanding

Document number: P1855R0

Date: 2019-10-05

Reply-to:

Ben Craig (National Instruments) <ben dot craig at gmail dot com>

Bryce Adelstein Lelbach (NVIDIA) <brycelelbach at gmail dot com>

Audience: Library Evolution Working Group

The problem

In Cologne, [the mothership landed](#) and `<coroutine>` got updated to use `op<=>`. `<coroutine>` is a [freestanding header](#). `<compare>` isn't listed as freestanding.

Pedantically, `<coroutine>` is part of the implementation, so it can do whatever it wants. However, if the user wants to compare `coroutine_handles`, then I think they need to include `<compare>` [\[expr.spaceship\]](#).

So we should add `<compare>` to freestanding to resolve the issue. This follows the C++98 - C++20 design precedent of having core language support in the freestanding library.

Proposed resolution:

Make `<compare>` freestanding. `<compare>` was identified as a potential addition to freestanding in the February 2018, Pre-Jacksonville mailing in the omnibus [Freestanding Proposal R1](#) paper. At the time, adding it to freestanding wasn't as urgent, as no other freestanding facilities depended on it.

The header `<compare>` specifies types, objects, and functions for use primarily in connection with the three-way comparison operator. This largely involves types with `constexpr`, `noexcept` comparison operator members. There are also some concepts, a `type_trait`-like struct, and a "Nieblويد" for the `compare_three_way` algorithm.

None of the facilities in `<compare>` require exceptions, RTTI, heap storage, virtual functions, function pointers, recursion, thread local storage, floating point, dynamic global initialization, dynamic global destructors, process exit functionality, atomics, or thread-safe statics.

None of the facilities in `<compare>` require facilities in non-freestanding headers.

`<compare>` can be implemented entirely in a header, without requiring a built library that needs to be linked against.

Making `<compare>` freestanding this late in the standards process isn't ideal, but it is still very low risk.

Consequences of delaying or rejecting:

Suppose we decide not to put this in C++20, or decide not to do this at all. What are the ramifications for the standard? What are the ramifications to users?

In the abstract, the standard will be falsely advertising that `coroutine_handle` can be compared or used with the three-way comparison operator in freestanding. There will also be a core language feature that isn't readily

usable in freestanding.

In practice, there aren't many users of strictly minimal freestanding implementations. Implementers are allowed to provide additional facilities beyond the bare minimum, and would likely provide a `<compare>` header. Even if the implementer didn't provide `<compare>`, it would only affect the ability of the user to do three-way comparisons and ordered comparisons on `coroutine_handle`.

Adding the header to the freestanding subset later wouldn't break binary or source compatibility.

Wording:

This wording is relative to [N4830](#).

1. In 16.5.1.3 [\[compliance\]](#), add a new row to [Table 22](#):

Table 22 — C++ headers for freestanding implementations

Subclause		Header(s)
[...]	[...]	[...]
17.10 [support.initlist]	Initializer lists	<code><initializer_list></code>
17.11 [cmp]	Comparisons	<compare>
17.12 [support.cooutine]	Coroutines support	<code><coroutine></code>
[...]	[...]	[...]